

Inference-Scale Complexity in ANN-SNN Conversion for High-Performance and Low-Power Applications

Tong Bu^{1,3†}, Maohua Li^{2†}, Zhaofei Yu^{1,3*}

[†] Equal contribution, ^{*} Corresponding author

putong30@pku.edu.cn, maohuali@hhu.edu.cn, yuzf12@pku.edu.cn

Abstract

Spiking Neural Networks (SNNs) have emerged as a promising substitute for Artificial Neural Networks (ANNs) due to their advantages of fast inference and low power consumption. However, the lack of efficient training algorithms has hindered their widespread adoption. Even efficient ANN-SNN conversion methods necessitate quantized training of ANNs to enhance the effectiveness of the conversion, incurring additional training costs. To address these challenges, we propose an efficient ANN-SNN conversion framework with only inference scale complexity. The conversion framework includes a local threshold balancing algorithm, which enables efficient calculation of the optimal thresholds and fine-grained adjustment of the threshold value by channel-wise scaling. We also introduce an effective delayed evaluation strategy to mitigate the influence of the spike propagation delays. We demonstrate the scalability of our framework in typical computer vision tasks: image classification, semantic segmentation, object detection, and video classification. Our algorithm outperforms existing methods, highlighting its practical applicability and efficiency. Moreover, we have evaluated the energy consumption of the converted SNNs, demonstrating their superior low-power advantage compared to conventional ANNs. This approach simplifies the deployment of SNNs by leveraging open-source pre-trained ANN models, enabling fast, low-power inference with negligible performance reduction. Code is available at <https://github.com/putshua/Inference-scale-ANN-SNN>.

1. Introduction

Recent advancements in large models have significantly reshaped the landscape of deep learning technology, industry, and the research community. These advanced models, characterized by their massive scale and unprecedented zero-

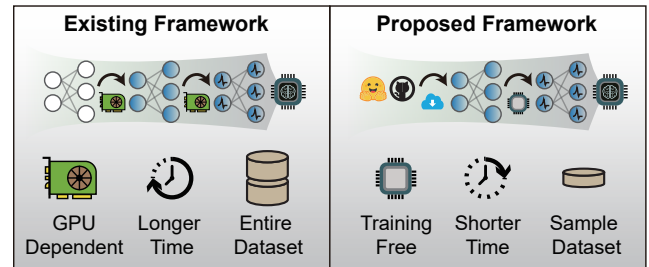


Figure 1. Compared to existing frameworks that require retraining a quantized ANN, the proposed framework is able to directly convert a pre-trained ANN to an SNN at inference-scale complexity, significantly reducing computational requirements, minimizing dependence on GPUs, and requiring only a small subset of the original dataset.

shot generalizability in downstream tasks, greatly impact our daily lives. Nevertheless, the pursuit of larger models raises concerns about high energy consumption for model inference and training. The deployment of large models on resource-constrained devices has also become a challenge.

Spiking neural networks (SNNs), a well-established type of neural network that mimic the function of biological neurons and are considered the third generation of ANNs [44], offer a potential solution to these issues as an alternative to Artificial Neural Networks (ANNs). Unlike ANNs, spiking neurons encode information as discrete events (binary spikes or action potentials) and generate outputs based on both their inputs and internal state. This unique processing mechanism enables SNNs to effectively handle spatio-temporal information while leveraging sparse representations, leading to more efficient and biologically plausible information processing.

With the development of the neuromorphic computing hardware [10, 11, 17, 41, 42, 49, 50, 68, 69], SNNs can now be deployed on neuromorphic chips and further applied in power-limited scenarios [9, 56]. However, the lack of efficient training algorithms has hindered their widespread application. Recent learning methods have made significant progress in training deep convolutional SNNs [15, 26] and large spiking transformers using supervised learning algorithms [57, 67, 70]. While supervised training enables SNNs

¹ Institute for Artificial Intelligence, Peking University

² School of Artificial Intelligence and Automation, Hohai University

³ School of Computer Science, Peking University

to achieve performance comparable to ANNs with the same architecture and reduces inference time, the memory and time costs for training scale linearly with the number of inference time-steps, making it impractical for training large energy-efficient models. A more feasible approach is ANN-to-SNN conversion [6], which transforms pre-trained ANNs into SNNs with minimal computational overhead. However, converted SNNs typically require more inference time-steps to match the performance of their ANN counterparts, leading to increased latency and higher energy consumption. A potential solution is to re-train a modified ANN before conversion, which can significantly enhance SNN performance while reducing inference time-steps [4, 29]. Although retraining-based conversion methods generally improve performance across most tasks, the additional computational cost of re-training poses a notable overhead.

In this paper, we propose an efficient post-training ANN-SNN conversion algorithm that directly obtains high-performance SNNs from pre-trained ANNs with inference-scale complexity. Figure 1 illustrates the key difference between existing ANN-to-SNN conversion frameworks and our proposed conversion framework. Compared to training-required methods, our approach greatly reduces the need for GPU resources, requires fewer data samples, and shortens the overall conversion process. We employ a local-learning-rule based algorithm for rapid conversion from ANNs to SNNs, avoiding the requirements for computationally expensive backpropagation-based training or fine-tuning. Additionally, we introduce a delayed evaluation strategy to further enhance performance. This conversion framework simplifies the deployment of SNNs by leveraging open-source, pre-trained ANN models, allowing direct application of converted SNNs on neuromorphic hardware with only inference-level computational costs. Overall, the proposed conversion framework offers two potential advantages. First, the lightweight local learning algorithm reduces dependence on GPUs, enabling online mapping from ANN to SNN on neuromorphic chips. Second, this approach reduces the need for extensive retraining of models, making it highly suitable for the practical deployment of large models in resource-constrained environments. Our contribution can be summarized as follows:

- We introduce an inference-scale complexity ANN-SNN conversion framework, enabling the conversion of pre-trained ANN models into high-performance SNNs while maintaining computational costs at inference level or even lower.
- We propose an efficient ANN-SNN conversion framework, theoretically deriving an upper bound for the conversion error between the original ANN and the converted SNN. To minimize this error bound, we introduce a light-weight local learning method for threshold optimization. Additionally, we design a delayed evaluation strategy to miti-

gate the influence of spike propagation delays.

- We experimentally demonstrate that our method is a lightweight, plug-and-play solution capable of integrating with various conversion methods for further performance improvement. Our framework successfully scales to three typical computer vision tasks, outperforming existing ANN-SNN conversion methods in classification tasks while also demonstrating its practical applicability in handling regression-based vision tasks.

2. Related Works

Researchers have endeavored to explore effective SNN training approaches for decades. Early studies mainly focused on the unsupervised Spike-Timing-Dependent Plasticity (STDP) algorithm [7, 25]. These studies aimed to utilize the STDP algorithm for shallow networks and feature extractors [13, 34, 46, 59, 62]. In parallel, supervised training approaches have been developed to achieve high-performance SNNs. Bohte et al. [1] were pioneers in applying the back-propagation algorithm to train SNNs, introducing a temporal-based backpropagation method that uses timestamps as intermediate variables during gradient computation. Wu et al. [64] treated SNNs as recurrently connected networks and utilized Back Propagation Through Time (BPTT) for training, termed spatial-temporal backpropagation (STBP). Unlike these methods, Cao et al. [6] proposed an ANN-SNN conversion method that achieves high-performance SNNs from pre-trained ANNs. Currently, research efforts are primarily focused on enhancing the efficiency of training algorithms for high-performance SNNs, leveraging the three methods mentioned above.

Based on whether global backpropagation is required, SNN learning methods can be ascribed into post-training and training-involved approaches. Training-required methods, including direct training of SNNs [17, 18, 32, 48, 58, 64–66], re-training before ANN-SNN conversion [5, 12, 23, 27, 31, 38, 63] and fine-tuning of converted SNNs [2, 37, 51, 52], typically require the participation of the back-propagation algorithms, making them more computationally intensive and resource-demanding. Post-training learning methods mainly refer to conversion-based algorithms that directly convert the pre-trained ANN into SNN. Cao et al. [6] pioneered this approach by mapped the weights of a lightweight CNN network to an SNN, demonstrating significant energy efficiency improvements through hardware analysis. Diehl et al. [14] proposed a weight and threshold balancing method to mitigate performance degradation, while Hunsberger and Eliasmith [28] further extended conversion-based learning to LIF neurons. Rueckauer et al. [54] established a theoretical framework for conversion-based algorithms, leading to further optimizations such as robust normalization, an enhanced threshold balancing method designed to optimize the trade-off between inference latency and accuracy [55].

Han and Roy [20] extended ANN-to-SNN conversion to temporal coding SNNs, improving accuracy and introducing a reset-by-subtraction mechanism instead of the traditional reset-to-zero, thereby preventing information loss [21]. Beyond classification tasks, conversion-based methods have been successfully applied to regression problems such as object detection, with Kim et al. [35] demonstrating the feasibility of spike-based object detection models. Further optimizations include adding an initial membrane potential for improved accuracy and reduced latency [4, 22], as well as employing a lightweight conversion pipeline to optimize thresholds and introduce a bias term at each inference step [36]. Although performance is limited, those lightweight training methods are able to acquire SNNs from pre-trained ANNs with negligible cost of multiple iterations of inference.

3. Preliminaries

3.1. Neuron Models

The core components of conventional ANNs are point neurons, where their forward propagation can be represented as a combination of linear transformations and non-linear activations, that is

$$\mathbf{a}^l = A(\mathbf{w}^l \mathbf{a}^{l-1}). \quad (1)$$

Here, $\mathbf{w}^l$ is the weights between layer $l - 1$ and layer l , $\mathbf{a}^l$ is the activation vector in layer l , and $A(\cdot)$ denotes the non-linear activation function.

The neurons in SNNs are spiking neurons with temporal structures. Like most ANN-SNN conversion methods [3, 5, 6, 12, 14, 55], we employ the Integrate-and-Fire (IF) neuron [19, 30] in this paper. For computational convenience, we use the following discrete neuron function:

$$\mathbf{v}^l(t^-) = \mathbf{v}^l(t-1) + \mathbf{w}^l \mathbf{s}^{l-1}(t) \theta^{l-1}, \quad (2)$$

$$\mathbf{s}^l(t) = H(\mathbf{v}^l(t^-) - \theta^l), \quad (3)$$

$$\mathbf{v}^l(t) = \mathbf{v}^l(t^-) - \theta^l \mathbf{s}^l(t). \quad (4)$$

Here $\mathbf{s}^l(t)$ describes whether neurons in layer l fire at time-step t , with firing triggered when the membrane potential before firing $\mathbf{v}^l(t^-)$ reaches the firing threshold θ^l , as represented by the Heaviside step function $H(\cdot)$. To alleviate information loss, we adopt the "reset-by-subtraction" mechanism [21, 53] for updating the membrane potential $\mathbf{v}^l(t)$ after spike emission. Specifically, instead of resetting to the resting potential, the membrane potential is reduced by θ^l after emitting a spike.

3.2. ANN-SNN Conversion

The pivotal idea of ANN-SNN conversion is to map the activation of analog neurons to the postsynaptic potential

(or firing rate) of spiking neurons. Specifically, by defining the initial membrane potential as $\mathbf{v}^l(0)$ and accumulating the neuron function (Equations 2 and 4) from 1 to T and dividing both sides by time-step T , we obtain:

$$\frac{\mathbf{v}^l(T) - \mathbf{v}^l(0)}{T} = \frac{\sum_{t=1}^T \mathbf{w}^l \mathbf{s}^{l-1}(t) \theta^{l-1} - \sum_{t=1}^T \mathbf{s}^l(t) \theta^l}{T}. \quad (5)$$

This equation reveals a linear relationship between the average firing rate of neurons in adjacent layers by defining average postsynaptic potential as $\mathbf{r}^l(t) = \sum_{i=1}^t \mathbf{s}^l(i) \theta^l / t$:

$$\mathbf{r}^l(T) = \mathbf{w}^l \mathbf{r}^{l-1}(T) - \frac{\mathbf{v}^l(T) - \mathbf{v}^l(0)}{T}. \quad (6)$$

As $T \rightarrow +\infty$, the conversion error can generally be assumed to approach zero. Therefore, using the equations presented, a trained ANN model can be converted into an SNN by replacing ReLU neurons with IF neurons, which forms the fundamental principle of ANN-SNN conversion. Since Equations 1 and 6 are not mathematically identical, some conversion error typically remains. To mitigate these errors, threshold balancing [14] and weight scaling algorithms [36, 55] play a crucial role. Both techniques aim to adjust synaptic weights or neuron thresholds according to the distribution range of neuron inputs, effectively reducing clipping errors. Previous studies have shown that weight scaling and threshold balancing are equivalent in effect and can achieve high-performance SNNs [4].

4. Methods

In this section, we present our post-training ANN-SNN conversion framework. We begin by defining the conversion error and deriving its upper bound. After that, we propose a local threshold balancing method based on a local learning rule. Additionally, we introduce the channel-wise threshold balancing technique, the pre-neuron max pooling layer, and the delayed evaluation strategy. These techniques are seamlessly integrated into a comprehensive conversion framework to enhance both performance and efficiency.

4.1. Conversion Error Bound

We follow the conversion error representation from [5] and define the conversion error e^l in layer l as the 2-norm of the difference between the postsynaptic potential of neurons in SNNs and the activation output of neurons in ANNs, that is

$$e^l = \|\mathbf{S}(\hat{\mathbf{z}}^l; \theta^l) - \mathbf{A}(\mathbf{z}^l)\|_2. \quad (7)$$

Here, $\hat{\mathbf{z}}^l = \mathbf{w}^l \mathbf{r}^{l-1}$ denotes the average input current from layer $l - 1$ in SNNs, and $\mathbf{z}^l = \mathbf{w}^l \mathbf{a}^{l-1}$ represents the activation input from layer $l - 1$ in ANNs. Both the ANN and SNN models share the same weights, denoted as $\mathbf{w}^l$. $\mathbf{A}(\mathbf{z}^l)$ represents the output of nonlinear ReLU activation function while

$S(\hat{z}^l; \theta^l) = \mathbf{r}^l(T)$ represents the average output synaptic potential at time-step T with given average input $\hat{z}^l$.

Since our primary objective is to minimize the error of the final outputs, we define the conversion error between the ANN and SNN models as the layer-wise conversion error in the last layer L , denoted as $e_{\text{model}} = e^L$. A straightforward approach would be to directly use the conversion error between models as a loss function and optimize it, which can be viewed as a variant of knowledge distillation [66]. However, this method is computationally expensive in terms of both time and memory, resembling the challenges of supervised training of SNN. To address this, we scale the conversion error and derive an error upper bound, thereby simplifying the optimization process. We formalize this approach in the following theorem.

Theorem 1 *The layer-wise conversion error can be divided into intra-layer and inter-layer errors:*

$$e^L \leq \underbrace{\|S(\hat{z}^L; \theta^L) - A(\hat{z}^L)\|_2}_{\text{intra-layer error}} + \underbrace{\|\mathbf{w}^L\|_2 e^{L-1}}_{\text{inter-layer error}}. \quad (8)$$

Given that both ANN and SNN models receive the same input in the first layer, leading to $e^0 = 0$, the upper bound for the conversion error between ANN and SNN models in an L -layer fully-connected network is given by

$$e_{\text{model}} = e^L \leq \sum_{l=1}^L \left(\prod_{k=l+1}^L \|\mathbf{w}^k\|_2 \right) \|S(\hat{z}^l; \theta^l) - A(\hat{z}^l)\|_2 \quad (9)$$

Here $\|\mathbf{w}^k\|_2$ is the matrix norm ($p=2$) or spectral norm of the weight matrix. The detailed proof is provided in the Appendix. Theorem 1 indicates that the conversion error is bounded by the weighted sum of errors across all layers. Based on this result, we introduce the local threshold balancing algorithm in the following section.

4.2. Local Threshold Balancing Algorithm

The target of ANN-SNN conversion is to minimize the conversion error and achieve high-performance SNNs. An alternative approach is to optimize the error bound, defined as:

$$\arg \min_{\theta} \mathbb{E}_{x^0 \in \mathcal{D}} \left[\sum_{l=1}^L \left(\prod_{k=l+1}^L \|\mathbf{w}^k\|_2 \right) \|S(\hat{z}^l; \theta^l) - A(\hat{z}^l)\|_2 \right]. \quad (10)$$

Here θ represents the set of threshold values to be optimized across all layers, i.e., $\theta = \{\theta^1, \theta^2, \dots, \theta^L\}$. x^0 denotes the input data sample drawn from the dataset $\mathcal{D}$. This approach leverages data-driven conversion, minimizing the expectation of the conversion error bound over the data distribution.

To reduce computational costs during optimization, we introduce two key simplifications. Firstly, we use a greedy strategy to optimize the threshold θ layer by layer, that is

For each layer l , $\arg \min_{\theta^l} \mathbb{E}_{x^0 \in \mathcal{D}}$

$$\left[\left(\prod_{k=l+1}^L \|\mathbf{w}^k\|_2 \right) \|S(\hat{z}^l; \theta^l) - A(\hat{z}^l)\|_2 \right]. \quad (11)$$

Secondly, we simplify the function $S(\cdot; \theta)$ by neglecting its temporal dynamics and approximating it with the clipping function $C(\cdot; \theta)$, as previous works have demonstrated that spiking neuron and clipping functions behave equivalently given sufficient number of time-steps [12]. The average input in the SNN can therefore be estimated as $\hat{z}^0 = z^0$; $\hat{z}^{l+1} = \mathbf{w}^{l+1} C(\hat{z}^l; \theta^l)$. Furthermore, We utilize the squared norm in the objective function for smoother optimization. The optimization problem then becomes:

For each layer l , $\arg \min_{\theta^l} \mathbb{E}_{x^0 \in \mathcal{D}}$

$$\left[\left(\prod_{k=l+1}^L \|\mathbf{w}^k\|_2 \right) \|C(\hat{z}^l; \theta^l) - A(\hat{z}^l)\|_2^2 \right], \quad (12)$$

$$\text{where } C(x; \theta) = \min(\max(0, x), \theta). \quad (13)$$

Intuitively, we can employ the stochastic gradient descent algorithm to optimize the threshold. Since $\prod_{k=l+1}^L \|\mathbf{w}^k\|_2$ can be considered as a constant when weights are fixed for each layer, this term can be absorbed into the learning rate. The final update rule for the local threshold balancing algorithm at each step is:

$$\Delta \theta^l = - \sum_{i=0}^{N-1} 2(\hat{z}_i^l - \theta^l) H(\hat{z}_i^l - \theta^l), \quad (14)$$

$$\theta^l \leftarrow \theta^l - \eta \Delta \theta^l. \quad (15)$$

Detailed derivation is provided in the Appendix. Here $\Delta \theta^l$ denotes the step size during optimization and η is the learning rate or update step size. N is the number of elements in the input vector, and $\hat{z}_i$ denotes each element in the vector $\hat{z}^l$. $H(\cdot)$ is the Heaviside step function.

In practice, as shown in Figure 2, we jointly optimize the threshold for each layer by sampling a small set of image samples from the training dataset. Before starting the conversion process, we first replace all ReLU activation functions $A(\cdot)$ with clipping functions $C(\cdot)$ and initialize the threshold to zero. We then randomly sample data batches from the training dataset and perform model inference with these data samples. The thresholds are locally updated during the inference until convergence. Notably, the parameter θ^l is updated locally, ensuring that the computational cost of each iteration is similar to the inference of the ANN model and the computational cost of the whole conversion process is at inference level.

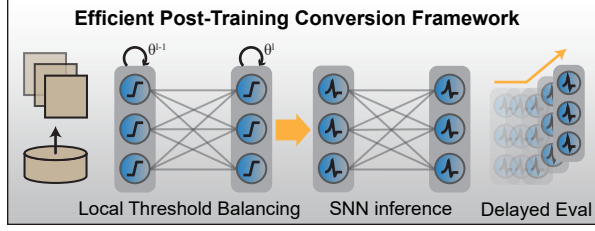


Figure 2. Illustration of the proposed conversion framework. It requires only a small subset of data for conversion using local threshold balancing. During inference, the delayed evaluation technique is employed to enhance the accuracy of output estimation.

4.3. Pre-Neuron Max Pooling Layer

The conversion of the max pooling layer presents a significant challenge. As a pivotal feature in the process of down-sampling input representations, the max pooling layer is a fundamental component in convolutional neural networks and is widely used in most convolutional architectures. However, due to the binary nature of spiking neurons, max pooling layers cannot effectively perform downsampling in feature maps. Typically, in a single max-pooling operation, multiple elements may share the same value, preventing the extraction of the most salient features. Consequently, max pooling is often avoided as a downsampling layer in SNNs. In most previous ANN-SNN methods, the max pooling layer in ANN is often replaced by an average pooling layer, and the model is re-trained or fine-tuned before conversion.

To convert architectures that contain max pooling layers, we propose a simple yet effective method. We replace all max-pooling layers with neuron layers, allowing the pre-neuron floating-point input to pass through the downsampling layer before being input into the neurons. We have the following theorem:

Theorem 2 *The order of the max pooling layer and ReLU layer does not affect the ANN output results.*

$$\max_i R(z) = R(\max_i z), \text{ when } \max(z) > 0. \quad (16)$$

Here $R(\cdot)$ denotes the ReLU activation. The detailed proof is provided in the Appendix. Theorem 2 guarantees that such an operation does not affect the final performance. When deployed on neuromorphic chips or FPGAs, the max pooling operation can be efficiently implemented through a comparator, avoiding additional floating-point multiplications and minimizing power consumption.

4.4. Channel-wise Threshold Balancing

During conversion, following Kim et al. [35], Li et al. [36], we employ a channel-wise local threshold balancing method applicable to both fully connected and convolutional layers.

In this approach, neurons in each channel share a single threshold for convolutional layers while each neuron has an

independent threshold for fully connected layers. Specifically, for convolutional layers, we channel-wisely determine the threshold value, while for fully connected layers, we optimize the threshold element-wise. The update function is given by:

$$\Delta\theta_c^l = - \sum_{i=0}^{N-1} 2(\hat{z}_{c,i}^l - \theta_c^l) H(\hat{z}_{c,i}^l - \theta_c^l), \quad (17)$$

where c denotes the channel index (or element index for fully connected layers) and i denotes the element index within the current channel.

After optimization, the thresholds for each channel can be individually absorbed into the corresponding channel of the convolutional kernel or the weight vectors of the fully connected layers. This channel-wise (or element-wise) operation ensures that the behavior of the IF neuron remains unaffected. This algorithm aims to more precisely determine the optimal threshold value while preserving the fundamental properties of spiking neurons.

4.5. Delayed Evaluation Strategy

As introduced in [4], the first spike from the last layer often takes longer than expected to appear due to propagation delays between layers. This delay creates significant fluctuations in the firing rate during the initial time-steps, as no spikes are generated until the first one reaches each layer [29]. To address this issue, we propose an effective delayed step selection strategy building upon the delayed evaluation technique [29].

As shown in Figure 2, the basic idea of the delayed evaluation is to compute the average spike count in the last layer from time-step t_0 to T , where t_0 denotes the delayed step, rather than averaging over all time-steps. Since the output is collected from the average spike count in the interval $[t_0, T]$ at the last layer, we can rewrite Equation 6 for the last layer as

$$\mathbf{r}^L(t_0, T) = \mathbf{w}^L \mathbf{r}^{L-1}(t_0, T) - \frac{\mathbf{v}^L(T) - \mathbf{v}^L(t_0)}{T - t_0}, \quad (18)$$

where $\mathbf{r}^L(t_0, t) = \sum_{i=t_0}^t \mathbf{s}^L(i) \theta^L / (t - t_0)$ denotes the average postsynaptic potential from time t_0 to t , and L represents the last layer index. When $T \gg t_0$, one can still guarantee that the basic equivalence of the average postsynaptic potential and ANN activations holds.

In our strategy, we do not start counting the average output spike until $\tilde{t}_0$, where $\tilde{t}_0$ is the expected spike timing of the first spike in the last layer. We estimate $\tilde{t}_0$ based on the converted SNN model, denoted as

$$\tilde{t}_0 = \sum_{l=1}^L \frac{\theta^l - v^l(0)}{\max_i (R(\hat{z}^l))} \approx \sum_{l=1}^L \frac{\theta^l - v^l(0)}{\max_i (R(\mathbf{z}^l))}. \quad (19)$$

Here $\max_i(\cdot)$ selects the maximum element of a given vector. The notation $\overline{(\cdot)} = \mathbb{E}_{x^0 \in \mathcal{D}}[(\cdot)]$ represents the expectation of the given input over dataset. Additionally, we use the ReLU function $R(\cdot)$ to prevent negative estimated time. Before evaluation, we first calculate $\tilde{t}_0$ according to Equation 19 and consider it as an indicator for the delayed evaluation strategy. In practice, if the inference step T is less than $\tilde{t}_0 + 4$, we set $t_0 = T - 4$ and collect the average output over the time interval $[T - 4, T]$. If T exceeds $\tilde{t}_0 + 4$, we use the interval $[\tilde{t}_0, T]$ for output estimation. The value 4 is empirically chosen to ensure a sufficient delay while allowing adequate time for averaging spikes over time.

This technique introduces minimal additional training cost and does not alter the model’s inference process, making it seamlessly integrated into the proposed conversion framework without disrupting the overall pipeline.

5. Experiments

We conduct extensive experiments to demonstrate the effectiveness of our method and highlight its potential practical value. We first showcase the advantages of our approach by integrating it with existing conversion frameworks, comparing it with state-of-the-art post-training conversion algorithms, and conducting ablation experiments to validate the effectiveness of the whole conversion pipeline. Subsequently, we apply our algorithm to various visual tasks, including image classification, semantic segmentation, object detection, and video classification. The feasibility of our algorithm is highlighted on both classification and regression tasks, showcasing its generalizability across different datasets and its superiority over existing conversion algorithms.

To underscore that our conversion method does not require additional training, we utilize open-source pre-trained ANN models for conversion, with most pre-trained sourced from TorchVision [45]. Additionally, we incorporate the membrane potential initialization algorithm [5], setting the membrane potential to half of the threshold before inference. The pseudocode for our conversion pipeline and detailed training settings are provided in the Appendix.

5.1. Plug-and-Play Integration for Enhanced Performance

In this section, we aim to highlight that our proposed conversion method provides a lightweight, plug-and-play solution that can be seamlessly integrated with other conversion algorithms to enhance performance. We select two representative ANN-SNN conversion approaches from existing work: the post-training fine-tuning (calibration) conversion method LCP/ACP [36, 37], and the training-required conversion method QCFS [5]. We reproduce the conversion pipeline of both methods, incorporate a local threshold balancing method for threshold searching, and use our delayed evaluation strategy. The evaluation is conducted on the ImageNet

Method	ANN	CF	Inference Step				
			32	64	128	256	512
LCP	75.66	×	50.21	63.66	68.89	72.12	74.17
		✓	53.80	67.18	72.18	74.07	74.56
ACP	75.66	×	64.54	71.12	73.45	74.61	75.35
		✓	65.95	72.87	74.21	74.73	75.26
QCFS	73.30	×	64.43	71.51	73.25	73.67	73.76
		✓	71.94	73.06	73.50	73.49	73.54

Table 1. Integration of the proposed conversion approach with existing framework

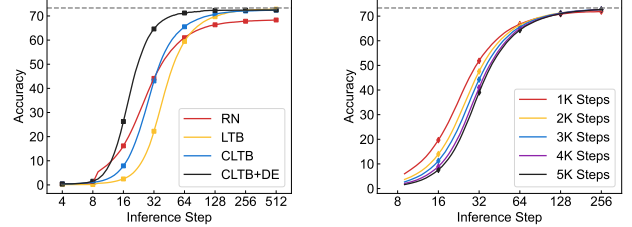


Figure 3. (a) Results on evaluating the impact of different techniques within the conversion framework. (b) Effect of iteration steps on SNN performance after conversion.

dataset with the ResNet-34 architecture.

The results, as detailed in Table 1, showcase the performance of various configurations. The column labeled "CF" indicates whether the proposed method is used. Notably, the composite approaches, which combine our method with either LCP/ACP or QCFS, consistently outperform the individual conversion method across most timesteps. After incorporating our methods, both QCFS and LCP exhibit performance improvements exceeding 3% when $T=32$, while maintaining consistent performance as T increases. This demonstrates the advantages of using our approach in conjunction with existing conversion techniques.

5.2. Ablation Study for the Conversion Framework

We demonstrate the effectiveness of our proposed conversion framework by using the ResNet-34 architecture on the ImageNet dataset. An ablation study is conducted to evaluate the impact of different techniques within the framework. Specifically, as shown in Figure 3a, we compare the performance of four different SNNs obtained by combinations of different techniques. The baseline SNN is obtained using the Robust-Norm [54] algorithm (RN, red curve), which is a commonly used post-training ANN-SNN conversion method that sets the 99-th percentile activation value as the threshold. The other SNNs are obtained by using the layer-wise local threshold balancing algorithm (LTB, yellow curve), the channel-wise threshold balancing algorithm (CLTB, blue curve), and a combination of channel-wise threshold balancing and delayed evaluation (CLTB+DE, black curve).

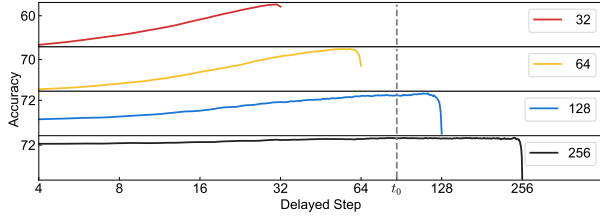


Figure 4. Effect of delayed steps

Figure 3a presents the accuracy changes of the converted SNNs with respect to the increasing time-steps. The performance of the SNN obtained from the full conversion pipeline (black curve) consistently outperforms the other SNNs across all time-steps, demonstrating an excellent balance between fast inference and high performance of the proposed conversion framework. This is particularly evident when combined with the proposed delayed evaluation technique. Compared to the robust norm method (red curve), the peak performance of the SNNs obtained by the proposed local threshold balancing method (black, blue and yellow curves) more closely aligns with the original ANN accuracy. Furthermore, the use of the channel-wise threshold balancing (CLTB, blue curve) results in a noticeable performance improvement over the vanilla threshold balancing (LTB, yellow curve) algorithm.

5.3. Effect of the Iteration Step

We evaluate the influence of the iteration steps of the local threshold balancing algorithm. The iteration number of the local threshold balancing serves as the hyper-parameter of the method. We focus on the most straightforward approach by empirically evaluating the impact of different iteration steps. We converted five different SNNs from the pre-trained ImageNet ResNet-34 model, varying the iterations steps from 1000 to 5000. The batch size during local threshold balancing is consistently set to 100. Figure 3b shows the final accuracy of the different converted SNNs. As number of iterations increases, the peak performance of the converted SNNs at different inference steps approaches the accuracy of the original ANN. Furthermore, these SNNs demonstrate better performance at low time-steps when fewer iteration steps are used. On the ImageNet dataset, with only 1000 iterations of local threshold update, the performance gap between the SNN at 512 inference time-steps and the ANN is approximately 1%. With 5000 iterations, this difference is further reduced to around 0.2%. Notably, the accuracy surpasses 65% within just 55 time-steps, achieving a balance between inference time and performance.

5.4. Effect of the Delayed Step

We conduct experiments to demonstrate the effectiveness of the proposed delayed strategy on the ImageNet dataset using the ResNet-34 architecture. As shown in Figure 4, we evaluate the impact of the delayed step across four different

Architecture	Method	ANN	Inference Step				
			32	64	128	256	≥ 512
Post-Training							
ResNet-34 ¹	RMP	70.64	-	-	-	55.65	60.08
ResNet-34 ²	LCP	75.66	55.16	67.56	72.48	74.53	75.44
MobileNet	LCP	73.40	0.20	15.47	55.95	66.35	72.19
MobileNetV2	Ours	71.88	19.39	49.93	64.31	68.89	70.48
ResNet-18 ³	Ours	69.76	65.55	68.53	69.03	69.21	69.21
ResNet-34 ³	Ours	73.31	64.88	71.17	72.30	72.42	72.46
Training-involved							
ResNet-34 ³	QCFS	73.30	64.43	71.51	73.25	73.67	73.76
ResNet-34 ²	ACP	75.66	64.65	71.30	73.94	75.00	75.45
ResNet-34 ³	Ours*	73.30	71.94	73.06	73.50	73.49	73.54

¹ Standard ResNet-34 architecture without batch normalization.

² Two more layers in the first conv-block compared to standard ResNet-34.

³ Standard ResNet-18/34 architecture.

Table 2. Comparison of the proposed method and previous works on the ImageNet dataset.

inference time-step settings: 32 steps (red curve), 64 steps (yellow curve), 128 steps (blue curve), and 256 steps (black curve). The grey vertical line indicates the estimated t_0 for the currently converted SNN. For each sub-figure, the x-axis represents the number of delayed steps, while the y-axis denotes the accuracy.

Although the delayed strategy is pre-determined before inference, the estimated t_0 can still serves as an indicator that provides a near-optimal strategy to select the delayed steps. When the total inference step is less than the estimated t_0 (red and yellow curves), the final performance gradually improves with the increase in the delayed steps, but drops significantly when the delayed step approaches the inference step. In the situation that the total inference step exceeds t_0 (blue and black curve), the performance first increases then fluctuates when delayed step is larger than t_0 , and ultimately experiences a sharp decline as the delayed steps get close to the inference step. The above results indicate that we can always reach near-optimal performance with the proposed delayed evaluation strategy.

5.5. Test on Image Classification task

We evaluate our conversion method on the classification task with the ImageNet dataset, employing different architectures including ResNet and MobileNet. To highlight the advantages of the proposed conversion algorithm, we conduct comparisons with previous post-training algorithms, showcasing its superior performance. When using the ResNet-34 architecture, the SNN converted by our algorithm exceeds 71% accuracy within 64 time steps. In contrast, the SNNs converted by the previous algorithm [20] and the calibration-required conversion algorithm [37] require longer time-steps to achieve comparable accuracy. We also implement the

proposed conversion framework on QCFS pre-trained models (denote as "Our*" in Table 2), and compare with the sota training-involved ANN-SNN techniques. The accuracy of the proposed method reaches 71.94% within 32 steps, demonstrating significant advantages compared to the fine-tuning required advanced calibration technique ACP [37] and training-involved vanilla QCFS [5]. The experiments on the ImageNet dataset demonstrate the superior performance of the proposed conversion algorithm and highlight its potential for large-scale applications.

In addition, we demonstrate the energy-saving advantage of our method by theoretically estimating the energy efficiency of our converted SNN. We use the same energy estimation approach as Bu et al. [5]. With ResNet-34 architecture on ImageNet, SNN converted from our algorithm can achieve energy efficiency at 622FPS/W while maintaining 90% performance. In comparison, the energy efficiency of the original ANN is only 22FPS/W, which is 28 times lower than that of the converted SNN. For detailed energy consumption analysis, please refer to the Appendix.

5.6. Test on Semantic Segmentation, Object Detection, and Video Classification Tasks

We further extend the proposed ANN-SNN conversion method to semantic segmentation, object detection, and video classification tasks. For the semantic segmentation task, we evaluate our method on two commonly used datasets: Pascal VOC 2012 [16] augmented by SBD [24] and MS COCO 2017 [39]. We employ various semantic segmentation models, including FCN [43] and DeepLabV3 [8], provided by TorchVision [45] and other open-source repositories. As shown in Table 3, our method is compatible with tackling semantic segmentation tasks without any preparatory training. Specifically, we achieved a 53.50% mIoU with the FCN model using a ResNet-34 backbone in 64 time-steps on the Pascal VOC dataset and a 61.52% mIoU with the DeepLabV3 model using a ResNet-50 architecture in 128 timesteps on the more complex MS COCO 2017 dataset. Given that pixel-level classification tasks require precise model output, this excellent performance highlights the effectiveness and generalizability of the proposed method.

For object detection tasks, we use the benchmark dataset MS COCO 2017 [39] with open-source models from TorchVision [45]. We employ the fully convolutional object detection method RetinaNet [40] and FCOS [60], and convert the pre-trained ANN models to SNNs using the proposed method. Table 3 shows detailed performance under different inference time-steps. Within 64 time-steps, the converted SNN can achieve 27.3% mAP with RetinaNet and 26.5% mAP with FCOS, which represents a significant improvement compared to previous explorations [35, 47]. The previous method required over 1000 time-steps to achieve comparable performance as ANN, which is detrimental to

Dataset	Arch.	ANN	Inference Step			
			32	64	128	256
Semantic Segmentation (mIoU%)						
PascalVOC	FCN-18	47.30	43.49	47.02	47.26	47.23
	FCN-34	54.99	46.60	53.50	54.75	54.76
	DeepLab-18	55.75	48.59	53.91	55.40	55.60
	DeepLab-34	58.95	41.20	55.74	58.36	58.68
MS COCO	FCN-50	60.67	28.66	50.38	57.01	59.53
	DeepLab-50	63.01	39.02	56.62	61.52	62.61
Object detection (mAP%)						
MS COCO	RetinaNet-50	36.4	16.6	27.3	32.2	34.3
	FCOS-50	39.2	12.2	26.5	32.1	33.6
Video Classification (Acc%)						
S-Kinetics-400	MC3-18	63.52	55.21	61.60	62.43	62.73
	R3D-18	62.56	53.46	60.31	61.12	61.15

Table 3. Performance on semantic segmentation (mIoU%), object detection (mAP%), and video classification (Acc%) tasks.

real-time detection and energy efficiency.

For the video classification tasks, we use a small Kinetics-400 dataset [33], which is a subset of the original dataset. More specifically, we split the original validation set into two independent sets, one for our threshold balancing procedure, accounting for about 60% of the original validation set, and the other for the test. As a result, the new train set contains 12000 videos and the new test set contains 7881 videos. The converted MC3-18 [61] SNN model achieves 61.60% accuracy within 64 time-steps, showing great potential of our method in more complex computer vision tasks.

6. Conclusion and Limitation

This paper introduces the concept of a lightweight post-training ANN-SNN conversion algorithm with inference-scale computational cost, which can directly convert pre-trained ANN models into SNNs without GPU-based training. The significant reduction in training overhead highlights the potential of the rapid deployment of large-scale SNNs in various scenarios and makes the instant on-chip conversion from pre-trained ANN to SNN possible. However, there is a trade-off between performance and training cost, with the overall performance of the proposed post-training framework being slightly weaker compared to other training-involved SNN learning methods. Additionally, while this work focuses on the conversion of convolutional SNNs, future efforts will aim to extend such a conversion approach to transformer architectures and other complex layers.

7. Acknowledgement

This work is funded by National Natural Science Foundation of China (62422601, 62176003, 62088102), Beijing Municipal Science and Technology Program (Z241100004224004) and Beijing Nova Program (20230484362, 20240484703).

References

- [1] Sander M Bohte, Joost N Kok, and Johannes A La Poutré. Spikeprop: backpropagation for networks of spiking neurons. In *European Symposium on Artificial Neural Networks*, 2000. 2
- [2] Velibor Bojkovic, Srinivas Anumasa, Giulia De Masi, Bin Gu, and Huan Xiong. Data driven threshold and potential initialization for spiking neural networks. In *International Conference on Artificial Intelligence and Statistics*, pages 4771–4779. PMLR, 2024. 2
- [3] Romain Brette, Michelle Rudolph, Ted Carnevale, Michael Hines, David Beeman, James M Bower, Markus Diesmann, Abigail Morrison, Philip H Goodman, Frederick C Harris, et al. Simulation of networks of spiking neurons: a review of tools and strategies. *Journal of Computational Neuroscience*, 2007. 3
- [4] Tong Bu, Jianhao Ding, Zhaofei Yu, and Tiejun Huang. Optimized potential initialization for low-latency spiking neural networks. In *AAAI Conference on Artificial Intelligence*, 2022. 2, 3, 5
- [5] Tong Bu, Wei Fang, Jianhao Ding, PengLin Dai, Zhaofei Yu, and Tiejun Huang. Optimal ANN-SNN conversion for high-accuracy and ultra-low-latency spiking neural networks. In *International Conference on Learning Representations*, 2022. 2, 3, 6, 8
- [6] Yongqiang Cao, Yang Chen, and Deepak Khosla. Spiking deep convolutional neural networks for energy-efficient object recognition. *International Journal of Computer Vision*, 2015. 2, 3
- [7] Natalia Caporale and Yang Dan. Spike timing-dependent plasticity: a hebbian learning rule. *Annual Review of Neuroscience*, 31:25–46, 2008. 2
- [8] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. Rethinking atrous convolution for semantic image segmentation. *arXiv preprint arXiv:1706.05587*, 2017. 8
- [9] Yanqi Chen, Zhaofei Yu, Wei Fang, Zhengyu Ma, Tiejun Huang, and Yonghong Tian. State transition of dendritic spines improves learning of sparse spiking neural networks. In *International Conference on Machine Learning*, 2022. 1
- [10] Mike Davies, Narayan Srinivasa, Tsung-Han Lin, Gautham Chinya, Yongqiang Cao, Sri Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Imam, Shweta Jain, et al. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 2018. 1
- [11] Michael V DeBole, Brian Taba, Arnon Amir, Filipp Akopyan, Alexander Andreopoulos, William P Risk, Jeff Kusnitz, Carlos Ortega Otero, Tapan K Nayak, Rathinakumar Appuswamy, et al. TrueNorth: Accelerating from zero to 64 million neurons in 10 years. *Computer*, 2019. 1
- [12] Shikuang Deng and Shi Gu. Optimal conversion of conventional artificial neural networks to spiking neural networks. In *International Conference on Learning Representations*, 2021. 2, 3, 4
- [13] Peter U Diehl and Matthew Cook. Unsupervised learning of digit recognition using spike-timing-dependent plasticity. *Frontiers in Computational Neuroscience*, 9:99, 2015. 2
- [14] Peter U Diehl, Daniel Neil, Jonathan Binas, Matthew Cook, Shih-Chii Liu, and Michael Pfeiffer. Fast-classifying, high-accuracy spiking deep networks through weight and threshold balancing. In *International Joint Conference on Neural Networks*, 2015. 2, 3
- [15] Chaoteng Duan, Jianhao Ding, Shiyan Chen, Zhaofei Yu, and Tiejun Huang. Temporal effective batch normalization in spiking neural networks. In *Advances in Neural Information Processing Systems*, 2022. 1
- [16] M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman. The pascal visual object classes challenge: A retrospective. *International Journal of Computer Vision*, 2015. 8
- [17] Biao Fang, Yuhao Zhang, Rui Yan, and Huajin Tang. Spike trains encoding optimization for spiking neural networks implementation in fpga. In *International Conference on Advanced Computational Intelligence*, 2020. 1, 2
- [18] Wei Fang, Zhaofei Yu, Yanqi Chen, Timothée Masquelier, Tiejun Huang, and Yonghong Tian. Incorporating learnable membrane time constant to enhance learning of spiking neural networks. In *International Conference on Computer Vision*, 2021. 2
- [19] Wulfram Gerstner, Werner M Kistler, Richard Naud, and Liam Paninski. *Neuronal dynamics: From single neurons to networks and models of cognition*. Cambridge University Press, 2014. 3
- [20] Bing Han and Kaushik Roy. Deep spiking neural network: Energy efficiency through time based coding. In *European Conference on Computer Vision*, 2020. 3, 7
- [21] Bing Han, Gopalakrishnan Srinivasan, and Kaushik Roy. RMP-SNN: Residual membrane potential neuron for enabling deeper high-accuracy and low-latency spiking neural network. In *Computer Vision and Pattern Recognition*, 2020. 3
- [22] Zecheng Hao, Tong Bu, Jianhao Ding, Tiejun Huang, and Zhaofei Yu. Reducing ann-snn conversion error through residual membrane potential. *arXiv preprint arXiv:2302.02091*, 2023. 3
- [23] Zecheng Hao, Jianhao Ding, Tong Bu, Tiejun Huang, and Zhaofei Yu. Bridging the gap between anns and snns by calibrating offset spikes. In *International Conference on Learning Representations*, 2023. 2
- [24] Bharath Hariharan, Pablo Arbeláez, Lubomir Bourdev, Subhransu Maji, and Jitendra Malik. Semantic contours from inverse detectors. In *International Conference on Computer Vision*, 2011. 8
- [25] Donald Olding Hebb. *The organization of behavior: A neuropsychological theory*. Psychology press, 2005. 2
- [26] JiaKui Hu, Man Yao, Xuerui Qiu, Yuhong Chou, Yuxuan Cai, Ning Qiao, Yonghong Tian, Xu Bo, and Guoqi Li. High-performance temporal reversible spiking neural networks with O(1) training memory and O(1) inference cost. In *International Conference on Machine Learning*, 2024. 1
- [27] Zihan Huang, Xinyu Shi, Zecheng Hao, Tong Bu, Jianhao Ding, Zhaofei Yu, and Tiejun Huang. Towards high-performance spiking transformers from ann to snn conversion. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 10688–10697, 2024. 2

- [28] Eric Hunsberger and Chris Eliasmith. Spiking deep networks with LIF neurons. *arXiv preprint arXiv:1510.08829*, 2015. 2
- [29] Sungmin Hwang, Jeesoo Chang, Min-Hye Oh, Kyung Kyu Min, Taejin Jang, Kyungchul Park, Junsu Yu, Jong-Ho Lee, and Byung-Gook Park. Low-latency spiking neural networks using pre-charged membrane potential and delayed evaluation. *Frontiers in Neuroscience*, 2021. 2, 5
- [30] Eugene M Izhikevich. Which model to use for cortical spiking neurons? *IEEE Transactions on Neural Networks*, 2004. 3
- [31] Haiyan Jiang, Srinivas Anumasa, Giulia De Masi, Huan Xiong, and Bin Gu. A unified optimization framework of ann-snn conversion: towards optimal mapping from activation values to firing rates. In *International Conference on Machine Learning*, 2023. 2
- [32] Haiyan Jiang, Vincent Zoonekynd, Giulia De Masi, Bin Gu, and Huan Xiong. TAB: Temporal accumulated batch normalization in spiking neural networks. In *International Conference on Learning Representations*, 2024. 2
- [33] Will Kay, Joao Carreira, Karen Simonyan, Brian Zhang, Chloe Hillier, Sudheendra Vijayanarasimhan, Fabio Viola, Tim Green, Trevor Back, Paul Natsev, et al. The kinetics human action video dataset. *arXiv preprint arXiv:1705.06950*, 2017. 8
- [34] Saeed Reza Kheradpisheh, Mohammad Ganjtabesh, Simon J Thorpe, and Timothée Masquelier. STDP-based spiking deep convolutional neural networks for object recognition. *Neural Networks*, 2018. 2
- [35] Seijoon Kim, Seongsik Park, Byunggook Na, and Sungroh Yoon. Spiking-YOLO: Spiking neural network for energy-efficient object detection. In *AAAI Conference on Artificial Intelligence*, 2020. 3, 5, 8
- [36] Yuhang Li, Shikuan Deng, Xin Dong, Ruihao Gong, and Shi Gu. A free lunch from ANN: Towards efficient, accurate spiking neural networks calibration. In *International Conference on Machine Learning*, 2021. 3, 5, 6
- [37] Yuhang Li, Shikuan Deng, Xin Dong, and Shi Gu. Error-aware conversion from ann to snn via post-training parameter calibration. *International Journal of Computer Vision*, 2024. 2, 6, 7, 8
- [38] Yuhang Li, Tamar Geller, Youngeun Kim, and Priyadarshini Panda. SEENN: Towards temporal spiking early exit neural networks. *Advances in Neural Information Processing Systems*, 2024. 2
- [39] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision*, 2014. 8
- [40] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *International Conference on Computer Vision*, 2017. 8
- [41] Zibin Liu, Banglei Guan, Yang Shang, Shunkun Liang, Zhenbao Yu, and Qifeng Yu. Optical flow-guided 6dof object pose tracking with an event camera. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 6501–6509, 2024. 1
- [42] Zibin Liu, Banglei Guan, Yang Shang, Qifeng Yu, and Laurent Kneip. Line-based 6-dof object pose estimation and tracking with an event camera. *IEEE Transactions on Image Processing*, 2024. 1
- [43] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Computer Vision and Pattern Recognition*, 2015. 8
- [44] Wolfgang Maass. Networks of spiking neurons: the third generation of neural network models. *Neural Networks*, 1997. 1
- [45] TorchVision maintainers and contributors. Torchvision: Pytorch’s computer vision library. <https://github.com/pytorch/vision>, 2016. 6, 8
- [46] Timothée Masquelier and Simon J Thorpe. Unsupervised learning of visual features through spike timing dependent plasticity. *PLoS Computational Biology*, 2007. 2
- [47] Joaquín Royo Miquel, Silvia Tolu, Frederik ET Schöller, and Roberto Galeazzi. Retinanet object detector based on analog-to-spiking neural network conversion. In *International Conference on Soft Computing & Machine Intelligence*, 2021. 8
- [48] Emre O Neftci, Hesham Mostafa, and Friedemann Zenke. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 2019. 2
- [49] Nicolas Perez Nieves and Dan F. M. Goodman. Sparse spiking gradient descent. In *Advances in Neural Information Processing Systems*, 2021. 1
- [50] Jing Pei, Lei Deng, Sen Song, Mingguo Zhao, Youhui Zhang, Shuang Wu, Guanrui Wang, Zhe Zou, Zhenzhi Wu, Wei He, et al. Towards artificial general intelligence with hybrid tianjic chip architecture. *Nature*, 2019. 1
- [51] Nitin Rath and Kaushik Roy. DIET-SNN: A low-latency spiking neural network with direct input encoding and leakage and threshold optimization. *IEEE Transactions on Neural Networks and Learning Systems*, 2021. 2
- [52] Nitin Rath, Gopalakrishnan Srinivasan, Priyadarshini Panda, and Kaushik Roy. Enabling deep spiking neural networks with hybrid conversion and spike timing dependent backpropagation. In *International Conference on Learning Representations*, 2020. 2
- [53] Bodo Rueckauer, Iulia-Alexandra Lungu, Yuhuang Hu, Michael Pfeiffer, and Shih-Chii Liu. Conversion of continuous-valued deep networks to efficient event-driven networks for image classification. *Frontiers in Neuroscience*, 2017. 3
- [54] Bodo Rueckauer, Iulia-Alexandra Lungu, Yuhuang Hu, Michael Pfeiffer, and Shih-Chii Liu. Conversion of continuous-valued deep networks to efficient event-driven networks for image classification. *Frontiers in Neuroscience*, 2017. 2, 6
- [55] Abhronil Sengupta, Yuting Ye, Robert Wang, Chiao Liu, and Kaushik Roy. Going deeper in spiking neural networks: VGG and residual architectures. *Frontiers in Neuroscience*, 2019. 2, 3
- [56] Jiangrong Shen, Qi Xu, Jian K. Liu, Yueming Wang, Gang Pan, and Huajin Tang. ESL-SNNs: An evolutionary structure learning strategy for spiking neural networks. In *AAAI Conference on Artificial Intelligence*, 2023. 1

- [57] Xinyu Shi, Zecheng Hao, and Zhaofei Yu. Spikingresformer: Bridging resnet and vision transformer in spiking neural networks. In *Computer Vision and Pattern Recognition*, 2024. 1
- [58] Sumit B Shrestha and Garrick Orchard. Slayer: Spike layer error reassignment in time. *Advances in Neural Information Processing Systems*, 2018. 2
- [59] Sen Song, Kenneth D Miller, and Larry F Abbott. Competitive hebbian learning through spike-timing-dependent synaptic plasticity. *Nature Neuroscience*, 3(9):919–926, 2000. 2
- [60] Zhi Tian, Chunhua Shen, Hao Chen, and Tong He. FCOS: fully convolutional one-stage object detection. In *International Conference on Computer Vision*, 2019. 8
- [61] Du Tran, Heng Wang, Lorenzo Torresani, Jamie Ray, Yann LeCun, and Manohar Paluri. A closer look at spatiotemporal convolutions for action recognition. *CoRR*, abs/1711.11248, 2017. 8
- [62] John J Wade, Liam J McDaid, Jose A Santos, and Heather M Sayers. Swat: A spiking neural network training algorithm for classification problems. *IEEE Transactions on Neural Networks*, 2010. 2
- [63] Xiaofeng Wu, Velibor Bojkovic, Bin Gu, Kun Suo, and Kai Zou. Ftbc: Forward temporal bias correction for optimizing ann-snn conversion. In *European Conference on Computer Vision*, pages 155–173. Springer, 2024. 2
- [64] Yujie Wu, Lei Deng, Guoqi Li, Jun Zhu, and Luping Shi. Spatio-temporal backpropagation for training high-performance spiking neural networks. *Frontiers in Neuroscience*, 2018. 2
- [65] Yujie Wu, Lei Deng, Guoqi Li, Jun Zhu, Yuan Xie, and Luping Shi. Direct training for spiking neural networks: Faster, larger, better. In *AAAI Conference on Artificial Intelligence*, 2019.
- [66] Qu Yang, Jibin Wu, Malu Zhang, Yansong Chua, Xinchao Wang, and Haizhou Li. Training spiking neural networks with local tandem learning. *Advances in Neural Information Processing Systems*, 2022. 2, 4
- [67] Man Yao, JiaKui Hu, Tianxiang Hu, Yifan Xu, Zhaokun Zhou, Yonghong Tian, Xu Bo, and Guoqi Li. Spike-driven transformer v2: Meta spiking neural network architecture inspiring the design of next-generation neuromorphic chips. In *International Conference on Learning Representations*, 2024. 1
- [68] Man Yao, Ole Richter, Guangshe Zhao, Ning Qiao, Yunnan Xing, Dingheng Wang, Tianxiang Hu, Wei Fang, Tugba Demirci, Michele De Marchi, et al. Spike-based dynamic computing with asynchronous sensing-computing neuromorphic chip. *Nature Communications*, 2024. 1
- [69] Friedemann Zenke and Emre O Neftci. Brain-inspired learning on neuromorphic substrates. *Proceedings of the IEEE*, 2021. 1
- [70] Rui-Jie Zhu, Qihang Zhao, Guoqi Li, and Jason K Eshraghian. Spikegpt: Generative pre-trained language model with spiking neural networks. *arXiv preprint arXiv:2302.13939*, 2023. 1